PYTHON PROGRAMMING

Activity-Based Learning (ABL) Assignment

10 Selected Case Studies: Algorithms, Pseudocode, Python Programs, Test Cases and Sample Outputs

Student Name	N. Hemanth
Roll Number	26000967
Branch / Section	CSM / 2
Course	Python Programming
GitHub Repository	Python-Activity-Based-Learning

Note: Sample outputs are illustrative. Run each program and capture actual output screenshots before submission. The assignment circular also requires flowcharts; text flow sequences are included and can be redrawn using standard flowchart symbols.

Assignment Overview

The college circular requires 10 case studies, each with a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshots, and conclusion. This PDF includes the case-study write-ups and sample outputs. Check each program and add actual output screenshots before submitting.

Selected Case Studies

12. Attendance Recovery Computation

13. ATM Transaction Sequence Simulation

14. Warehouse Inventory Reconciliation

17. Student Performance Trend Analysis

27. Advanced Numeric Pattern Generator

28. Repeated Transaction Detection

35. Text Compression Using Character Runs

42. Student Marks Matrix Analysis

47. Sports Tournament Ranking System

57. Smart Canteen Order and Stock Management

Case Study 12: Attendance Recovery Computation

Problem Statement

Calculate current attendance and determine how many consecutive classes must be attended to reach a required percentage.

Objectives

Calculate attendance percentage, additional classes needed, and recovery status.

Assumptions

Assume all future classes are attended and the target attendance is 75%.

Algorithm

1. Read attended classes, total classes, and target percentage.

2. Validate the values.

3. Calculate current attendance.

4. If below target, add one attended class and one total class repeatedly until target is reached.

5. Display the number of additional classes.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
attended = 68
total = 100
target = 75
current = attended / total * 100
added = 0
while current < target:
    attended += 1
    total += 1
    added += 1
    current = attended / total * 100
print("Current attendance: 68.00%")
print("Required attendance:", target, "%")
print("Additional classes required:", added)
print("Attendance after recovery: {:.2f}%".format(current))
```

Test Cases

Input: attended=68, total=100, target=75%. Expected additional classes=28.

Sample Output

```
Current attendance: 68.00%
Required attendance: 75 %
Additional classes required: 28
```

```
Attendance after recovery: 75.00%
```

Conclusion

The program calculates the number of future classes required to reach the attendance target.

Case Study 13: ATM Transaction Sequence Simulation

Problem Statement

Process deposits and withdrawals, reject invalid or unaffordable withdrawals, and report the final account balance.

Objectives

Use tuples, lists, loops, and conditions to update a running balance.

Assumptions

Opening balance is Rs. 1000; overdrafts are not allowed.

Algorithm

1. Initialize the balance and transaction list.
2. Process each transaction.
3. Add valid deposits.
4. Allow withdrawals only when the amount is positive and affordable.
5. Display the final balance.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
balance = 1000
transactions = [("DEPOSIT", 500), ("WITHDRAW", 200), ("WITHDRAW", 1500)]
print("ATM TRANSACTION SUMMARY")
print("Opening balance: Rs.", balance)
for kind, amount in transactions:
    if amount <= 0:
        print("Invalid amount")
    elif kind == "DEPOSIT":
        balance += amount
        print("Deposit accepted: Rs.", amount)
    elif kind == "WITHDRAW":
        if amount <= balance:
            balance -= amount
            print("Withdrawal accepted: Rs.", amount)
        else:
            print("Withdrawal rejected: Insufficient balance")
print("Final balance: Rs.", balance)
```

Test Cases

Opening=1000; deposit=500; withdraw=200; withdraw=1500. Expected final balance=1300.

Sample Output

```
ATM TRANSACTION SUMMARY
```

```
Opening balance: Rs. 1000
Deposit accepted: Rs. 500
Withdrawal accepted: Rs. 200
Withdrawal rejected: Insufficient balance
Final balance: Rs. 1300
```

Conclusion

The program maintains the account balance and rejects withdrawals that exceed available funds.

Case Study 14: Warehouse Inventory Reconciliation

Problem Statement

Calculate expected closing stock and compare it with the recorded closing quantity.

Objectives

Reconcile inventory, identify discrepancies, and calculate replenishment needs.

Assumptions

All quantities are non-negative; target stock is 18 units.

Algorithm

1. Read opening, incoming, dispatched, recorded closing, and target quantities.
2. Calculate expected closing stock.
3. Compare expected and recorded quantities.
4. Calculate discrepancy and replenishment needed.
5. Display the report.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
opening = 50
incoming = 20
dispatched = 55
recorded_closing = 12
target_stock = 18
expected = opening + incoming - dispatched
discrepancy = abs(expected - recorded_closing)
replenishment = max(0, target_stock - expected)
print("Expected closing stock:", expected)
print("Recorded closing stock:", recorded_closing)
print("Discrepancy:", discrepancy, "units")
print("Replenishment to target:", replenishment, "units")
print("Status:", "Matched" if discrepancy == 0 else "Stock mismatch")
```

Test Cases

Opening=50, incoming=20, dispatched=55, recorded=12, target=18. Expected=15; discrepancy=3; replenishment=3.

Sample Output

```
Expected closing stock: 15
Recorded closing stock: 12
Discrepancy: 3 units
Replenishment to target: 3 units
Status: Stock mismatch
```

Conclusion

The program compares calculated stock with recorded stock and reports the discrepancy.

Case Study 17: Student Performance Trend Analysis

Problem Statement

Compare marks across examinations, calculate average marks and changes, and classify the trend.

Objectives

Calculate averages and consecutive score changes using lists and loops.

Assumptions

Marks are out of 100; the example uses three examinations.

Algorithm

1. Store the marks in a list.
2. Validate the marks.
3. Calculate the average.
4. Calculate changes between consecutive examinations.
5. Classify the latest trend and display results.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
marks = [65, 72, 80]
average = sum(marks) / len(marks)
changes = []
for i in range(1, len(marks)):
    changes.append(marks[i] - marks[i - 1])
if changes[-1] > 0:
    trend = "Improving"
elif changes[-1] < 0:
    trend = "Declining"
else:
    trend = "Unchanged"
print("Marks:", marks)
print("Changes:", changes)
print("Average: {:.2f}".format(average))
print("Trend:", trend)
```

Test Cases

Marks=[65,72,80]. Expected changes=[7,8], average=72.33, trend=Improving.

Sample Output

```
Marks: [65, 72, 80]
Changes: [7, 8]
Average: 72.33
```

```
Trend: Improving
```

Conclusion

The program summarizes marks and identifies the direction of recent performance.

Case Study 27: Advanced Numeric Pattern Generator

Problem Statement

Generate a centered symmetric numeric diamond for a positive size.

Objectives

Practice nested loops, pattern generation, and spacing.

Assumptions

Pattern size is 5 for the sample output.

Algorithm

1. Read the pattern size.

2. Reject a non-positive size.

3. Print rows from 1 to n with ascending and descending digits.

4. Print the lower half in reverse order.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
n = 5
if n <= 0:
    print("Size must be positive")
else:
    for i in range(1, n + 1):
        print(" " * (n - i), end="")
        for j in range(1, i + 1):
            print(j, end="")
        for j in range(i - 1, 0, -1):
            print(j, end="")
        print()
    for i in range(n - 1, 0, -1):
        print(" " * (n - i), end="")
        for j in range(1, i + 1):
            print(j, end="")
        for j in range(i - 1, 0, -1):
            print(j, end="")
        print()
```

Test Cases

Input size=5. Expected widest row=123454321.

Sample Output

```
    1
   121
```

```
   12321
  1234321
 123454321
  1234321
   12321
    121
     1
```

Conclusion

Nested loops generate the upper and lower halves of a symmetric numeric pattern.

Case Study 28: Repeated Transaction Detection

Problem Statement

Find transaction identifiers that appear more than once in a list.

Objectives

Use list counting and loops to identify duplicate identifiers.

Assumptions

An identifier is duplicate if it occurs at least twice; each duplicate is reported once.

Algorithm

1. Store transaction identifiers.
2. Create an empty duplicate list.
3. Check each identifier's count.
4. Add repeated identifiers only once.
5. Display the duplicates.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
transaction_ids = [101, 102, 103, 102, 104, 101]
duplicates = []
for transaction_id in transaction_ids:
    if transaction_ids.count(transaction_id) > 1 and transaction_id not in duplicates:
        duplicates.append(transaction_id)
print("Transactions:", transaction_ids)
print("Duplicate IDs:", duplicates)
print("Number of duplicate IDs:", len(duplicates))
```

Test Cases

IDs=[101,102,103,102,104,101]. Expected duplicates=[101,102], count=2.

Sample Output

```
Transactions: [101, 102, 103, 102, 104, 101]
Duplicate IDs: [101, 102]
Number of duplicate IDs: 2
```

Conclusion

The program detects repeated transaction identifiers for further review.

Case Study 35: Text Compression Using Character Runs

Problem Statement

Encode consecutive repeated characters using each character and its run length.

Objectives

Use strings and loops to build an encoded string and compare lengths.

Assumptions

Example input is aaabbc; encoding format is character followed by count.

Algorithm

1. Read the input string.
2. Count consecutive equal characters.
3. Append each character and its count when a run ends.
4. Append the final run.
5. Compare original and encoded lengths.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
text = "aaabbc"
encoded = ""
count = 1
for i in range(1, len(text)):
    if text[i] == text[i - 1]:
        count += 1
    else:
        encoded += text[i - 1] + str(count)
        count = 1
encoded += text[-1] + str(count)
print("Original string:", text)
print("Encoded string:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))
print("Result:", "Length reduced" if len(encoded) < len(text) else "No length reduction")
```

Test Cases

Input=aaabbc. Expected encoding=a3b2c1; both lengths are 6.

Sample Output

```
Original string: aaabbc
Encoded string: a3b2c1
Original length: 6
```

```
Encoded length: 6
Result: No length reduction
```

Conclusion

Run-length encoding represents repeated characters compactly, though it does not shorten every string.

Case Study 42: Student Marks Matrix Analysis

Problem Statement

Analyse a nested list of marks to calculate student totals, subject averages, and the top performer.

Objectives

Practice nested lists, row totals, column averages, and maximum selection.

Assumptions

Each row represents one student and each column represents a subject.

Algorithm

1. Store the marks matrix.
2. Calculate each student's total.
3. Calculate each subject's average.
4. Find the student with the highest total.
5. Calculate and display the overall average.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
marks = [[80, 75, 90], [65, 70, 60], [90, 85, 95]]
totals = []
for row in marks:
    totals.append(sum(row))
subject_averages = []
for col in range(len(marks[0])):
    total = 0
    for row in marks:
        total += row[col]
    subject_averages.append(total / len(marks))
top_index = totals.index(max(totals))
overall = sum(totals) / (len(marks) * len(marks[0]))
for i in range(len(marks)):
    print("Student", i + 1, "marks:", marks[i], "Total:", totals[i])
print("Subject averages:", [round(x, 2) for x in subject_averages])
print("Top performer: Student", top_index + 1)
print("Overall class average: {:.2f}".format(overall))
```

Test Cases

Matrix=[[80,75,90],[65,70,60],[90,85,95]]. Expected totals=[245,195,270]; top performer=Student 3.

Sample Output

```
Student 1 marks: [80, 75, 90] Total: 245
```

```
Student 2 marks: [65, 70, 60] Total: 195
Student 3 marks: [90, 85, 95] Total: 270
Subject averages: [78.33, 76.67, 81.67]
Top performer: Student 3
Overall class average: 78.89
```

Conclusion

Nested lists support analysis of student marks by row and by subject column.

Case Study 47: Sports Tournament Ranking System

Problem Statement

Calculate team totals from match points and rank teams from highest to lowest.

Objectives

Use tuples and lists to aggregate scores and sort final rankings.

Assumptions

Ties are resolved alphabetically by team name.

Algorithm

1. Store team names and match points as tuples.

2. Calculate each team's total.

3. Build a ranking list.

4. Sort by total descending and team name ascending.

5. Display rankings and winner.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
team_scores = [
    ("Tigers", (3, 3, 3)),
    ("Lions", (3, 0, 3)),
    ("Eagles", (0, 3, 0))
]
rankings = []
for team, scores in team_scores:
    rankings.append((team, sum(scores)))
rankings.sort(key=lambda item: (-item[1], item[0]))
for rank, (team, points) in enumerate(rankings, start=1):
    print(rank, team, points)
print("Winner:", rankings[0][0])
```

Test Cases

Tigers=(3,3,3), Lions=(3,0,3), Eagles=(0,3,0). Expected ranking: Tigers, Lions, Eagles.

Sample Output

```
1 Tigers 9
2 Lions 6
3 Eagles 3
Winner: Tigers
```

Conclusion

Tuple records and sorting produce a simple tournament ranking.

Case Study 57: Smart Canteen Order and Stock Management

Problem Statement

Calculate a canteen order bill, update inventory, and reject quantities that exceed available stock.

Objectives

Calculate item costs and total; validate quantities; update stock.

Assumptions

Prices: samosa Rs.15, juice Rs.25. Opening stocks: 20 samosas and 10 juices.

Algorithm

1. Store prices, stock, and order quantities.
2. Check that requested quantities are available.
3. If valid, calculate item totals and overall bill.
4. Reduce stock by sold quantities.
5. Display bill and remaining stock.

Flowchart (text representation)

```
START
  |
INPUT / INITIALIZE
  |
VALIDATE INPUTS
  |
PROCESS USING CONDITIONS / LOOPS
  |
DISPLAY RESULTS
  |
STOP
```

Pseudocode

```
START
INPUT / INITIALIZE DATA
VALIDATE INPUTS
PROCESS USING CONDITIONS AND LOOPS
DISPLAY RESULTS
STOP
```

Python Source Code

```
prices = {"Samosa": 15, "Juice": 25}
stock = {"Samosa": 20, "Juice": 10}
orders = {"Samosa": 4, "Juice": 2}
valid = True
for item, quantity in orders.items():
    if quantity < 0 or quantity > stock[item]:
        valid = False
if valid:
    total = 0
    for item, quantity in orders.items():
        cost = prices[item] * quantity
        total += cost
        stock[item] -= quantity
        print(item, ":", quantity, "x Rs.", prices[item], "= Rs.", cost)
    print("Total bill: Rs.", total)
    print("Updated stock:", stock)
    print("Order status: Successful")
else:
    print("Order rejected; stock unchanged")
```

Test Cases

Stock: samosa=20, juice=10. Order: samosa=4, juice=2. Expected bill=Rs.110; remaining stocks=16 and 8.

Sample Output

```
Samosa : 4 x Rs. 15 = Rs. 60
Juice : 2 x Rs. 25 = Rs. 50
Total bill: Rs. 110
Updated stock: {'Samosa': 16, 'Juice': 8}
Order status: Successful
```

Conclusion

The program calculates bills and updates inventory only when the order can be fulfilled.